Campus Store Inventory System: Minimal Algorithm

1. Main Flow

1. Initialize an empty `inventory` collection and set a 5% tax rate constant.
2. Loop a menu that prompts the user for a choice (1-5) and links to the following sub-processes:
    - **Choice 1:** Call *Add Product*.
    - **Choice 2:** Call *Update Product*.
    - **Choice 3:** Call *View Inventory*.
    - **Choice 4:** Call *Purchase Products*.
    - **Choice 5:** Stop the loop and terminate the program.

2. Add Product

1. Prompt for the product name, stock quantity, and price.
2. Validate that stock and price are valid numbers and are not negative.
3. Map and save the data inside the `inventory` dictionary.

3. Update Product

1. Prompt for a target product name. Exit if it does not exist in the `inventory`.
2. Prompt for the new stock quantity and new price, checking for valid, non-negative numbers.
3. Overwrite the old stock and price values for that product.

4. View Inventory

1. Exit if the `inventory` is completely empty.
2. Loop through the collection and print a structured table of all products, stock levels, and prices.

5. Purchase Products & Generate Bill

1. Exit if no products exist. Otherwise, open a loop allowing the user to select valid product names and quantities to add to a temporary shopping `cart`.
2. Multiply quantities by unit prices to sum up the line totals and calculate the `subtotal`.
3. Deduct the purchased quantities directly from each product's remaining stock in the master `inventory`.
4. Calculate a discount against the `subtotal` based on spending tiers (15% for ≥$\boldsymbol{5000}$, 10% for ≥$\boldsymbol{2000}$, 5% for ≥$\boldsymbol{1000}$).
5. Compute the final payable amount: $\boldsymbol{Subtotal + 5\%\ Tax - Discount}$, and print the summary invoice receipt.

6. Scan all items in the store; print a "Low Stock Alert" warning for any product with a remaining count of 5 or fewer.